From Software Requirements Specifications to Agile User Stories Using Large Language Models: A PRISMA-Informed Systematic Evidence Review of Prompt Engineering, Quality Assessment, and Traceability

Zaher Fadhil Raham[1,2] · Sura Salah[3]

[1] College of Computing & Informatics, Universiti Tenaga Nasional (UNITEN), Malaysia | [2] Al-Nahrain University, Baghdad, Iraq

[3] Presidency of Al-Nahrain University, Baghdad, Iraq

Abstract

Background: Large language models (LLMs) are increasingly used to transform software requirements specifications (SRSs) and related requirements artifacts into Agile user stories and acceptance criteria. Their outputs must preserve source obligations while remaining clear, actionable, and testable. Objective: This review synthesizes source-verified studies on direct document-to-story conversion and closely related work on prompt engineering, story quality, and traceability. Methods: An auditable identification ledger consolidated 93 Scopus, 28 IEEE Xplore, and 86 ScienceDirect bibliographic entries (207 total). DOI and title reconciliation identified 26 cross-source duplicate entries, yielding 181 distinct candidate records. We organized the evidence by source-to-output task, study design, evaluation construct, and strength of direct applicability to SRS-to-story conversion. Results: Direct generation studies and complementary industrial and experimental investigations support the practical potential of LLM-assisted requirements transformation. Across studies, quality depends on the input artifact, prompt design, evaluation criterion, and human assessment. Sentence fluency and local semantic similarity are insufficient proxies for complete coverage or support by the originating SRS. Contribution: The synthesis proposes a source-grounded evaluation framework integrating obligation coverage, claim faithfulness, bidirectional traceability, story quality, acceptance-test readiness, robustness, and human revision effort. Conclusion: LLM-generated stories should be evaluated against explicit requirements and reviewed with human oversight before becoming authoritative backlog items.

Keywords: software requirements specification; Agile user stories; large language models; prompt engineering; source grounding; coverage; traceability; requirements quality.

1. Introduction

Software requirements specifications (SRSs) describe what a system is expected to do and the conditions under which it must operate. They may contain functional requirements, non-functional requirements, data constraints, interfaces, security rules, user roles and dependencies. In Agile development, the same project may be organized through shorter user stories and acceptance criteria. These artifacts have different purposes: an SRS is often intended to preserve an agreed body of requirements, whereas a user story supports negotiation, planning and incremental delivery. Transforming one into the other is therefore not simply summarizing a document.

Large language models (LLMs) offer an attractive interface for this conversion. A model can read natural-language requirements and produce role-oriented statements, typically in the form “As a [role], I want [capability], so that [benefit].” A model may also propose acceptance criteria, priorities, test scenarios or

backlog items. Yet the fact that an output looks like a user story does not establish that it is correct. A fluent story could lose an important restriction, assign an unauthorized role, merge multiple actions, or add a plausible but undocumented feature. These errors have practical consequences because downstream developers may treat generated stories as approved work items.

The user-story literature provides an established conceptual foundation. The Quality User Story (QUS) framework separates syntactic, semantic and pragmatic quality concerns; a syntactically valid statement may still be non-atomic, incomplete or inconsistent with its context [1]. User stories also support communication between stakeholder groups and act as boundary objects, so quality includes usefulness for shared understanding rather than only grammatical conformity [2]. Broader reviews of generative AI and software requirements engineering document expanding coverage of elicitation, analysis, validation, specification and traceability [3,4]. A prompt-engineering review similarly identifies many techniques, but warns that reproducibility and comparability remain limited [5].

Several primary studies now address different parts of this workflow. GeneUS investigates automated generation of user stories from requirements documents [8]. An industrial Agile model-driven development (MDD) study evaluates generated stories and related artifacts in real projects [10]. A study of interview-based generation and assessment directly examines whether LLMs can produce and judge user stories [9]. Other work assesses user-story refinement [15], generates acceptance criteria [12], or recovers requirements trace links [16]. A conceptual risk follows: results from interview-derived stories, goal-derived stories, story-to-task decomposition or issue-title-to-requirement generation are not interchangeable with tests of complete SRS-to-story transformation.

This review contributes a task-aware synthesis of reported evidence and a reproducible evaluation framework. It distinguishes direct SRS-to-story conversion from related generation and assessment tasks, compares the criteria used in the primary studies, and identifies the measurements needed to assess preservation of source obligations. Its methodological contribution is an operational joint assessment of source coverage, claim support, and traceability rather than a claim that automated story assessment is new.

2. Review Aim and Research Questions

The review asks what is known about LLM-assisted translation from existing software requirements artifacts to Agile user stories, and how confidently that knowledge can support practical adoption. It addresses five questions:

- RQ1. What source artifacts, LLM models and workflows have been used for requirements-to-user-story generation?
- RQ2. Which prompt-engineering strategies have been compared, and are their effects consistent across evaluation criteria?
- RQ3. How are user-story quality, source requirement coverage, claim faithfulness, acceptance criteria and traceability measured?
- RQ4. How have the studies designed datasets, comparators, human assessments and reproducibility procedures?
- RQ5. What evaluation protocol and research agenda would make future SRS-grounded transformation experiments reproducible and more trustworthy?

3. Review Methodology and Reporting Framework

3.1 Review design

This work uses a PRISMA-informed bibliographic identification and a structured synthesis of the available, source-verified research. The review questions and eligibility taxonomy were organized before evidence comparison. Direct SRS-to-user-story generation is treated separately from adjacent transformations, evaluation-method papers, and secondary reviews. Identification totals describe bibliographic candidates; thematic conclusions draw on primary-study reports with inspectable methods or outcomes and are interpreted in their original task context [19,20,26].

3.2 Literature identification and source material

The synthesis examines primary-study methods, datasets, model settings and reported outcomes by their actual source-to-output task. Details drawn from research reports or linked publisher records were recorded with a bibliographic identifier and a relevant source location. Because the evidence spans heterogeneous tasks and outcome definitions, results were not pooled numerically. Any illustration in this manuscript is distinguished from an experiment, and study-level statistics are not treated as a shared effect size.

The bibliographic ledger draws on a single Scopus search exported in two parts (93 entries, split 50 and 43), an IEEE Xplore export (28 entries), and a ScienceDirect citation collection with abstracts (86 entries). After reconciling title and DOI identifiers, the ledger retained 181 unique candidate records from 207 imported bibliographic entries. The 26 cross-source overlaps comprised 21 IEEE–Scopus matches and five further ScienceDirect matches. The breakdown of the resulting register is 93 Scopus records, seven IEEE-only records and 81 ScienceDirect-only records. Counts from the focused PDF collection are not combined with the bibliographic identification counts.

3.3 Eligibility logic and task taxonomy

Primary evidence is judged on relevance to the defined source-to-output transformation, rather than whether an article contains the words “LLM” and “user story.” Direct-task studies begin with an existing requirements document or clearly identified requirements material and generate Agile stories. Adjacent empirical studies may instead begin with stakeholder interviews, issue titles, goals, source code or pre-existing stories, or generate output artifacts other than stories. Such studies are retained as contextual evidence only when they inform prompting, evaluation or traceability. Purely unrelated AI applications, duplicates and earlier versions of identical experiments are excluded from counted study families. The latest published report is preferred when it subsumes an earlier preprint; versions are linked in the evidence register rather than independently counted.

We use four evidence tiers: (E1) direct requirements-document-to-story experiments; (E2) related but different input/output transformations; (E3) measurement and traceability methods; and (E4) broad secondary reviews or conceptual foundations. Claims about measured results draw on primary papers and preserve their task-specific denominators. Publisher abstracts and metadata support discovery but do not substitute for full-text extraction of detailed metrics.

3.4 Evidence extraction and synthesis

The extraction fields include bibliographic identity, input type, model and model version, prompting strategy, context handling, generated artifact, sample size, evaluation units, quality measures, human participation, numerical results, threats to validity, source page and direct-versus-adjacent evidence tier.

Results are checked against the PDF where available. Assertions that were corrected in the 32-claim audit were rewritten to preserve direction, denominator and task context. The evidence map is synthesized by construct because heterogeneous source inputs, models and metrics make pooled effect estimates inappropriate.

The review register preserves record identity, original source, search grouping, bibliographic identifiers, reviewer-specific title/abstract decisions and source verification notes. Model-assisted retrieval and prioritization are tracked independently from human screening. The evidence synthesis uses only findings that can be traced to the relevant primary report or verified publisher information; direct and adjacent tasks are reported separately, and extracted numerical outcomes retain the authors' original units and denominators.

The evidence register stores record identifiers, source groups, bibliographic metadata, deduplication matches and a structured source-link inventory. Screening judgments and their provenance are recorded separately from machine-assisted retrieval and source summaries. The companion research package publishes non-copyrighted metadata, verified-source links, the identification audit and a blank extraction/screening template. Licensed abstracts, publisher full texts and unapproved individual reviewer assessments are not redistributed.

3.6 PRISMA-informed identification flow

3.6 PRISMA-informed identification flow

Figure 1 depicts only the audited bibliographic identification stage. The Scopus export contributed 93 entries, IEEE Xplore 28 and a ScienceDirect citation collection 86, giving 207 imported records. Identifier/title reconciliation found 26 overlaps and produced 181 distinct bibliographic candidates. These are not the same as 181 eligible primary studies. Evidence synthesis is based on selected source-linked reports and makes no final included-study count from the candidate totals [26].

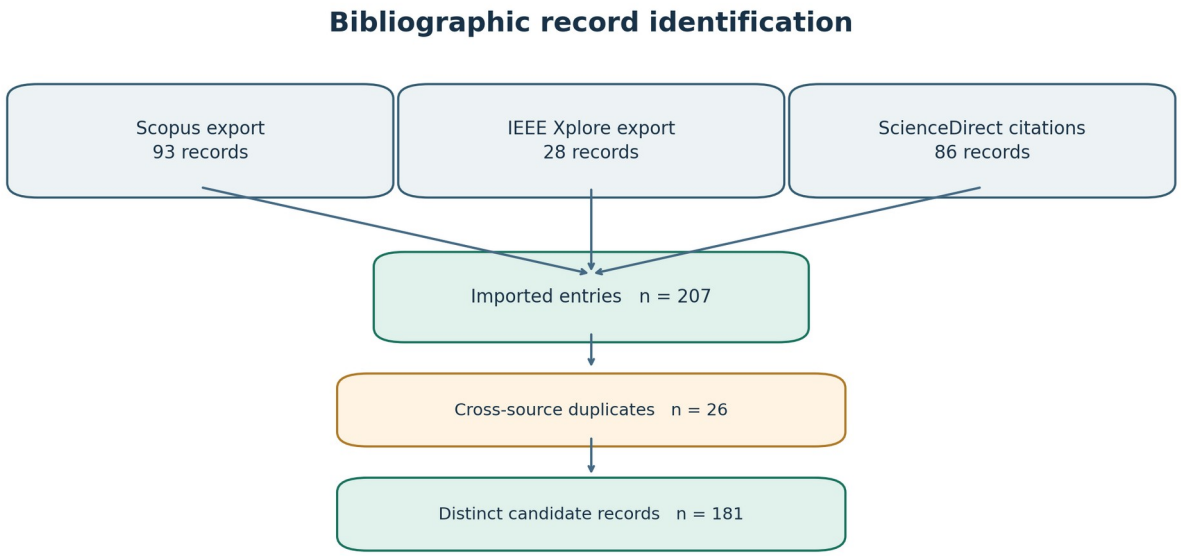


Figure 1. Bibliographic identification and duplicate reconciliation for the 181-record evidence register.

Table 1. Audited identification counts and source provenance.

Bibliographic identification element	Audited count	Documented source basis
Scopus export entries	93	Single Scopus search exported in two files (50 + 43)
IEEE Xplore export entries	28	21 entries also appear in the Scopus source group
ScienceDirect citation-list entries	86	Source citation collection with publisher abstracts
Imported bibliographic entries	207	93 + 28 + 86 imported bibliographic entries
Cross-source duplicate candidates	26	21 IEEE/Scopus + five additional ScienceDirect overlaps
Deduplicated candidate records	181	93 Scopus + seven IEEE-only + 81 ScienceDirect-only

3.7 Eligibility taxonomy and reviewer workflow

Eligibility is operationalized as follows. E1 (direct evidence) requires a software requirements document or comparable explicitly bounded requirements input, LLM-assisted generation of Agile user stories, and an inspectable evaluation. E2 covers adjacent transformations, including interview-to-story, issue-to-requirement, existing-story refinement, code-to-story, goal-to-story, or SRS regeneration. E3 captures measurement and traceability studies; E4 captures syntheses, methodological papers and foundational concepts. E2–E4 can inform the narrative and rubric but should not inflate the number of direct SRS-to-story experiments. Duplicate publications, linked preprints superseded by a final paper, and off-topic work are identified separately. Publication date and full-text availability should be verified before final exclusion. Reviews, tutorials and conceptual work may be retained as background but are not treated as independent primary empirical experiments.

The selection framework uses explicit task-direction and evidence-role coding. Title/abstract judgments distinguish Include, Exclude and Unclear, and preserve the source of each decision. Full-text appraisal focuses on the originating requirements artifact, LLM contribution, generated artifact, evaluation evidence, and relationship to the review questions. Related journal and conference versions are reconciled by bibliographic identity to avoid duplicate reporting. Studies that report only secondary synthesis or conceptual frameworks are used to explain background and evaluation constructs rather than counted as independent empirical experiments.

The register supports evidence-role classification across the 93 Scopus, seven IEEE-only and 81 ScienceDirect-only candidates. Relevant source-verified studies are then evaluated by the nature of their input, whether user stories are generated or merely assessed, whether claims are traceable to the source, and the level of empirical validation. Reported results are interpreted for their corresponding task, document type, domain, and study design. Figure 2 summarizes the analytical workflow.

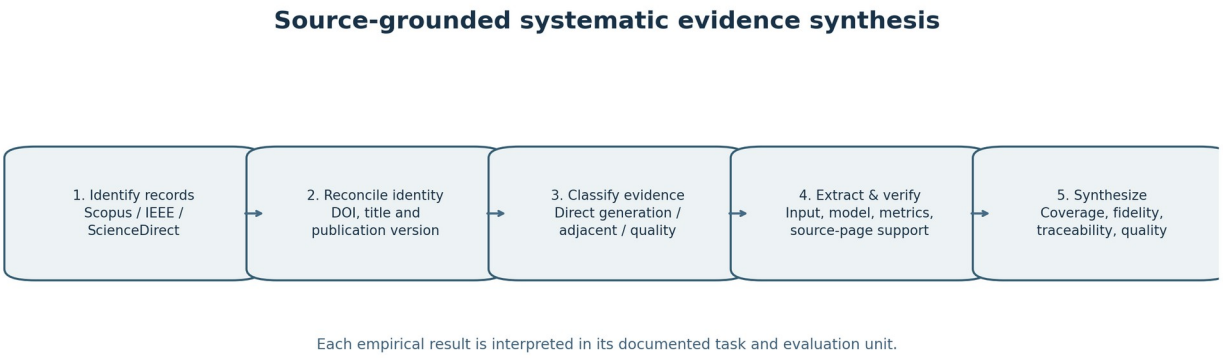


Figure 2. Task-aware identification, extraction, and synthesis workflow used in the review.

4. Conceptual Background: From Source Obligations to Agile Stories

4.1 Artifact semantics

SRS statements commonly vary in granularity. A single paragraph may define several functional operations, a performance threshold, an access-control rule and an exception path. The generation task should first identify **atomic obligations**—indivisible normative requirements that can be tested or checked—and then decide whether each obligation should become a story, an acceptance criterion, an explicit constraint, or a deferred item. A one-to-one mapping is not necessarily correct; one requirement may support several stories, and one story may consolidate multiple compatible requirements. The scientific task is to make those transformations inspectable.

A user story is not merely a shorter SRS sentence. Its role and benefit fields introduce assumptions that must be traced or explicitly marked as proposed. Acceptance criteria often carry the more precise boundary conditions required for testing. Security, privacy, performance, interoperability and regulatory obligations should not disappear because the output schema favors functional statements. In particular, non-functional requirements may attach to multiple stories and require a separate constraint registry.

4.2 Quality of a story versus faithfulness to the source

The QUS framework [1] provides criteria for well-formedness, atomicity, minimality and related qualities. The INVEST rubric used in practice has a different emphasis: Independent, Negotiable, Valuable, Estimable, Small and Testable. Both are relevant for whether a story is useful, but neither alone establishes that the story matches its originating source. A story can be clear and testable while introducing an invented reporting function. Conversely, a source-faithful story can still be too large or unclear. These outcomes must be measured separately to avoid hiding unsafe transformations behind average quality scores.

Traceability methods distinguish source identifiers from downstream artifact links. Pre-specification traceability research [7] and recent LLM-based link-recovery work [16] demonstrate that candidate relationships can be proposed automatically. A link prediction score is not the same construct as generation accuracy: a generated story may contain a correct cited source ID and still omit a separate required obligation, while a textually similar story may be unsupported by its cited source span.

5. Synthesis of the Research Evidence

5.1 Direct requirements-to-story generation

Rahman et al. introduced an automated approach to generating user stories from requirements material, known as GeneUS [8]. The workflow demonstrates how LLMs can produce structured user stories for downstream software-engineering use. Its findings are more nuanced than the statement that a particular prompt consistently "wins." The source audit found that a Refine-and-Thought approach performed better in several settings but not uniformly across measured criteria. For a reported user-story AlignScore comparison, a best-case normalized score favored that approach (0.99 versus 0.91 for the baseline), whereas a worst-case comparison favored the baseline (0.29 versus 0.40). Such extrema are not pooled means and cannot support universal ranking of prompts. The study also reports a developer questionnaire involving 76 participants; subjective developer assessments should be read alongside artifact-level accuracy rather than treated as an independent measure of complete requirements coverage.

Maha's thesis [24] extends the document-to-story line of work and is useful for describing staged and iterative prompting. Because it is a graduate thesis and overlaps conceptually with the GeneUS research program, it should be presented as contextual evidence rather than automatically counted as an independent peer-reviewed confirmation. The existing studies establish technical feasibility, but robust evidence that all original obligations are preserved across diverse SRS documents remains more limited than evidence that plausible stories can be generated.

5.2 Industrial relevance and out-of-scope content

Spijkman et al. investigated LLM-supported requirements engineering in Agile model-driven development and obtained practitioner evaluations in four industrial projects [10]. The generation tasks included stories, acceptance criteria and data models. Practitioners often appreciated the generated artifacts, but the source-based audit recorded that **25% of generated user stories were judged out of scope overall**, reaching **40% for one project**. The same paper reported a 74.6% macro-average positive response across two user-story validation statements. These observations are not contradictory: perceived usefulness and source conformity are separate properties. For SRS-to-story adoption, a positive satisfaction rating cannot remove the need for a requirement-level audit of unsupported additions.

The industrial setting is valuable because it exposes project context, vocabulary and stakeholder judgments that artificial benchmarks may omit. However, observed results from four projects in one organizational methodology should not be generalized to all requirements domains. A rigorous follow-up should record each document's length, requirement structure, role taxonomy, language, confidentiality requirements and accepted story granularity, and then evaluate whether gains persist on independent projects.

5.3 Story generation and assessment from other inputs

Quattrocchi et al. studied whether ten LLMs could elicit user stories through simulated customer interviews and assess story quality [9]. This work is a direct counterexample to any novelty claim that LLM-based user-story quality assessment has never been examined. Its central input is interaction-derived requirements rather than an authoritative complete SRS, so it does not by itself measure preservation of every obligation in a supplied source document. The study illustrates an important evaluator issue: grammatically sound stories and acceptance criteria do not guarantee diversity or complete stakeholder intent capture.

Hernández-Agüero et al. evaluated and refined 60 existing Spanish-language user stories against INVEST dimensions, comparing three human experts with GPT-5.1 in interface and API conditions [15]. The authors explicitly caution that their observational comparisons do not establish causal superiority or equivalence with expert judgment. This is relevant to evaluator calibration and repeated model runs, not a direct test of SRS-to-story source coverage. In the same way, SimAC explores collaborative LLM generation of acceptance criteria from user stories [12]; acceptance-criteria elaboration is a downstream activity that may improve testability while preserving the danger of adding unsupported details.

Other adjacent work reverses the direction of transformation, such as code-to-story generation, or decomposes stories into implementation tasks. These studies may contribute ideas for prompts, ranking and human feedback, but their input evidence differs substantially from natural-language SRS. Scores obtained on code snippets or issue titles should never be presented as estimates of how completely an LLM translates a requirements specification.

5.4 Missing-requirement identification and the coverage problem

Kabadi et al. tested whether LLMs can identify missing requirements from production-derived user-story sets [14]. Their three-domain comparisons reported higher recall for LLM-generated story sets than the original analyst sets in all evaluated domains. However, F1 did not favor the LLM across all domains: in the Trawelli case the analyst F1 was 84.2%, compared with 79.2% for the LLM. This is an example of a coverage–precision trade-off. It also exposes a reference-standard problem: a production-derived backlog can contain plausible additions not present in an initial requirements document, so matching the backlog may reward inference beyond a fixed source rather than strict preservation of that source.

An SRS-to-story study should therefore report at least two different outcomes. **Source coverage** asks how many original normative obligations are represented by supported generated items. **Unsupported-addition risk** asks how much generated content has no license in the provided source. These measures have different denominators and can move in opposite directions. Increasing coverage by creating more stories is not necessarily beneficial if it also produces many unsupported claims.

5.5 Source-grounded intermediate artifacts and reconstructability

ReqGenX [13] provides an important related example of controlling transformation provenance. It decomposes seven PURE requirements documents into source-grounded atoms, regenerates pre-SRS artifacts and reconstructs SRS content. The published abstract reports typical median atomic AlignScore values of 0.96–0.99, and downstream artifact values typically around 0.80–0.94. These are ranges for the authors' studied components, not a direct benchmark of generated Agile user stories. The study also shows variation in source coverage across documents and configurations despite high grounding scores for individual atoms. Thus, high local support does not prove that every source obligation has been retained.

The importance of ReqGenX for the present review is methodological: intermediate atomic statements and explicit provenance can make otherwise opaque transformations auditable. The PURE corpus itself is a public collection of real requirements documents [21]; it can support SRS-level experiments, but a separate collection of user stories cannot be treated as a matched ground-truth output unless the source-to-story pairing is independently established.

5.6 Prompt engineering and requirements-traceability methods

Prompt engineering in requirements engineering has been synthesized by Huang et al., who screened 867 records and included 35 primary studies [5]. Their taxonomy links prompting patterns to RE tasks and

supports more explicit and repeatable prompt design. Yet task-level heterogeneity limits any claim that one technique dominates in all settings. Paiva et al. compared two models and three prompting styles over 150 feature-request issue titles, yielding 900 generated requirements [11]. Their results show criterion-specific trade-offs: few-shot prompting increased mean singularity scores for both models, whereas an expert-style prompt was strong on verifiability. These were issue-title-to-requirement transformations, not complete SRS-to-story conversions. Prompt choice should therefore be treated as an empirical factor, not a guarantee of better source fidelity.

TraceLLM [16] evaluates prompt-guided trace-link recovery across four benchmark datasets and eight models. It reports benefits of demonstration selection and prompt refinement for requirements-to-downstream-artifact links. That evidence establishes that links can be recovered more effectively under certain configurations, not that a generated story has preserved every source obligation. Similarly, REQINONE [17] and consulting-company applications [18] show how LLMs can draft or reorganize requirements specifications, but their principal targets are not a complete controlled comparison of SRS-to-story coverage. These works remain important as evidence about constraints, context and requirements-document structure.

5.7 Convergent evidence across task families

Three conclusions are supported by the available heterogeneous studies. First, LLMs can generate plausible, often useful user stories and related requirements artifacts. Second, prompt effects and evaluator ratings depend on the chosen task, criterion, document and execution conditions. Third, local text quality and semantic similarity do not constitute an adequate proxy for preservation of all source obligations. These findings support a methodological contribution—a multidimensional evaluation protocol—rather than a claim that source-grounded transformation has never been studied.

5.8 Extracted characteristics of the principal studies

Table 2 summarizes representative studies that clarify direct generation, adjacent transformations and evaluation constructs. It is an explanatory task-level evidence table; the bibliographic identification denominator is not used as a study-level sample size or an estimate of effectiveness.

Table 2. Provisional study-level evidence extraction: input, dataset, LLM/prompt, metrics and human assessment.

Study	Input and output	Dataset	Model / prompt	Metrics	Human evaluation / boundary
GeneUS [8]	Requirements documents → user stories	Requirements material (number of independent SRSs not confirmed)	LLM; direct vs reasoning/structured prompting (RaT)	Normalized AlignScore; RUST evaluation	Developer-based evaluation reported by the study; task and sample scope require citation-specific interpretation

Study	Input and output	Dataset	Model / prompt	Metrics	Human evaluation / boundary
Spijkman et al. [10]	Industrial requirements → user stories/criteria/models	Four low-code warehouse-management projects	LLM with engineered reusable prompts; model version NR	Reported scope relevance and out-of-scope outputs; project-dependent	Project team practitioner ratings; Adjacent industrial setting
Quattrocchi et al. [9]	Simulated interviews → user stories / story-quality judgments	Emulated customer interviews	10 LLMs; evaluation guidance varies	Story quality, coverage/style/diversity, acceptance criteria	Expert / student comparisons, human assessment; Not a fixed SRS input
ReqGenX [13]	Legacy SRS → grounded atoms → pre-SRS artifacts → reconstructed SRS	Seven PURE SRS documents	Multi-LLM pipeline; constrained prompts, iterative refinement	AlignScore; Prometheus; information retention	Model-judge components; human protocol not confirmed here; Not direct story generation
Kabadi et al. [14]	Production-derived stories → missing-requirement discovery	Three industry domains/projects	LLM-based elicitation; prompt version NR	Recall, precision and F1 vs analyst baseline	Production story reference; human annotation details NR; Higher recall not always higher F1
Paiva et al. [11]	Issue titles → generated requirements	150 OSS feature-request titles; 900 generated requirements	Two LLMs × three prompting styles	Singularity, verifiability (quality framework)	Independent human validation NR in audit; Not user-story task
Hernández-Agüero et al. [15]	Existing Spanish user stories → assessment and refinement	60 existing stories	GPT-5.1 via interface and API; prompts NR	INVEST / expert comparison	Three human expert reviewers; Not generation from SRS
TraceLLM [16]	Artifact links → recovered traceability	Four benchmark datasets	Eight LLMs, demonstration / prompt strategies	Trace-link precision/recall/F1 as applicable	Human adjudication details NR here; Different trace-link targets
Goal2Story (F07/F13)	Goals/context → stories	StorySeek >1,000 goal-linked stories	Multi-agent small LLM fleet, context-sensitive prompting	FHR; QuACE	Human validation scope NR here; Not SRS-grounded benchmark
Design science low-code (F01)	Minimal functional requirements → story suggestions	Three low-code applications	LLM-based agent design process	Practitioner consistency/relevance	Five practitioner interviews; Small qualitative setting

5.9 Study quality and risk-of-bias considerations

Risk appraisal uses a domain-based signalling rubric rather than an unvalidated total score. D: dataset/sample representativeness and potential selection bias; C: adequacy of comparators and control conditions; E: independence and validity of evaluation, including LLM-as-judge leakage; R: reproducibility and release of prompts, data and annotations. Flags below mean an issue needing source verification or careful inference, not proof that the study is biased. A definitive quality appraisal requires full-text re-extraction and independent human adjudication.

Table 3. Verification-oriented quality matrix. All entries are provisional study-specific concerns; no standardized risk score or causal judgment is assigned.

Study	Dataset / selection	Comparators	Evaluation / bias	Reproducibility	Interpretation limit
GeneUS [8]	Sample scope to verify	Prompt conditions vary	Developer assessments of story candidates; no universal benchmark baseline	Source-setting and evaluation materials should be interpreted according to the original article	Mixed comparisons; not universal prompt advantage
Spijkman [10]	Four similar domain projects	No external randomized control	Practitioner judgment is contextual; no randomized effect is inferred	Industrial source confidentiality	Limited transfer to heterogeneous SRS
Quattrocchi [9]	Interview simulation, not real SRS	Models / prompts may vary	Human vs LLM judge comparability	Public artifacts to verify	Do not infer SRS fidelity
ReqGenX [13]	Seven PURE docs	Multi-stage workflow	Judge-derived measures; reconstruction	Code/data release to verify	Not direct story-generation outcome
Kabadi [14]	Three domains; production gold	Analyst baseline	Recall/F1 depend on reference	Industry transfer and leakage to verify	Recall gains do not imply all-metric wins
Paiva [11]	150 titles, not SRS	Two models × three prompts	Generated requirements quality judgments	Data/prompts availability to verify	Different input/output task
Hernández-Agüero [15]	60 Spanish stories	No causal baseline claimed	Three experts, GPT-5.1 comparisons	API/interface settings differ	Existing-story refinement only
TraceLLM [16]	Four link datasets	Eight models / example selection	Trace link correctness differs by type	Prompt/sample reuse to verify	No direct SRS→story demonstration

Table 5. Task-aware map of representative studies and evidence limits.

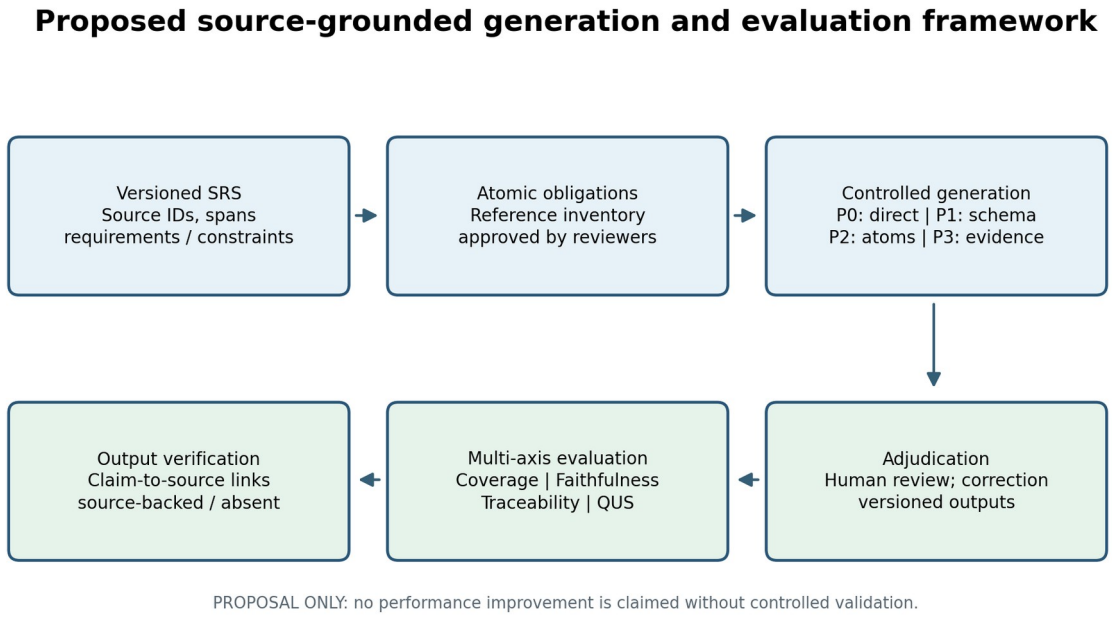
Study	Input → output	Main evidence	Why it is not interchangeable with complete SRS coverage
GeneUS / Rahman et al. [8]	Requirements document → user stories	Prompt comparisons, structured output, developer feedback	Metrics vary by case; story completeness must be independently checked
Spijkman et al. [10]	Industrial RE context → stories / criteria / models	Four project teams; relevance and scope evaluation	Practitioner usefulness coexists with out-of-scope output
Quattrocchi et al. [9]	Simulated customer interviews → stories / evaluation	Ten-model generation and quality-assessment study	No fixed authoritative SRS obligation inventory
Hernández-Agüero et al. [15]	Existing stories → ratings / refinements	Expert–model comparison under INVEST	Measures evaluation/refinement, not source document coverage
SimAC [12]	User stories → acceptance criteria	Collaborative LLM elaboration	May introduce acceptance details absent from original source

Kabadi et al. [14]	Production story references → elicited stories	Recall/F1 across three domains	Missing-requirement detection reference differs from SRS ground truth
ReqGenX [13]	SRS → atoms → artifacts → reconstructed SRS	Source grounding, reconstruction, coverage analysis	Not a direct SRS → Agile story benchmark
Paiva et al. [11]	Issue titles → requirements	900 outputs; prompt-criterion trade-offs	Input is short issue text rather than complete SRS
TraceLLM [16]	Requirements / design / tests → links	Prompt-guided traceability across benchmarks	Link prediction is not generative coverage or claim support

6. Proposed Source-Grounded Evaluation Framework

Our framework is a **proposed evaluation protocol**, not a previously validated model or a new empirical experiment. It defines explicit units of analysis and separates source evidence from generated text. A practical pipeline has six stages: (i) segment the original SRS; (ii) annotate atomic obligations and source spans; (iii) generate stories under controlled prompts; (iv) attach each output claim to its candidate source evidence; (v) independently judge support, coverage, quality and traceability; and (vi) log expert corrections and stable outputs. This architecture supports both human review and quantitative measurement.

Figure 3. Source-grounded generation and validation architecture (proposed, not empirically tested).



The generated artifact is a testable protocol; all quality claims require human adjudication.

6.1 Coverage and omissions

Let R denote the independently agreed set of applicable atomic source obligations and C the subset supported by at least one valid generated story or associated acceptance criterion. Source requirement coverage is |C|/|R|. This recall-like measure should separately identify functional obligations, non-functional constraints, exception paths and actor permissions. The denominator must be frozen before examining generated outputs, with a reasoned process for resolving ambiguous or overlapping source statements. A story that mentions a topic but omits its normative condition should not automatically count as coverage.

6.2 Source support and unsupported additions

Every generated story is segmented into auditable claims (actor, action, data entity, condition, benefit, constraint and acceptance-test statement). Reviewers judge each claim as **supported**, **contradicted**, **not stated**, or **indeterminate**, citing a specific source span and confidence rationale. Supported-claim proportion is the number of supported claims divided by the number adjudicated; unsupported-addition rate is the number judged "not stated" divided by the same denominator. Contradictions are reported separately because they represent a different risk from plausible additions. These are proposed measures and must not be confused with precision metrics used in earlier papers on different tasks.

6.3 Bidirectional traceability

A generated story may link to one or multiple source requirements. For each candidate link, an analyst records whether the linked source span genuinely supports the story claim. Trace-link precision, recall and F1 can be computed against a separately adjudicated reference link set, while requirement-level coverage checks whether each source obligation has at least one justified output link. A link is not considered valid solely because an LLM lists a requirement identifier; the cited passage and the assertion must agree. The framework also records one-to-many and many-to-one mappings and linked-source version IDs.

6.4 Story quality, testability and human correction

Story quality is independently scored with a prespecified QUS/INVEST-compatible rubric that covers role clarity, atomicity, ambiguity, negotiability, value, size and testability. Acceptance criteria are checked for observable conditions, boundary behavior and unsupported assumptions. At least two trained human reviewers independently assess a defined sample, then adjudicate disagreements. The evaluation reports agreement for the actual task and rating scale, along with expert editing minutes, number of rewritten claims, and post-edit acceptance decisions. Changes in fluency should not be described as improved fidelity without direct source checks.

6.5 Robustness and cost

Model/version, prompt text, decoding settings, output schema, context segmentation, retrieval sources, token budget and execution date must be preserved. Repeated runs are necessary because a single generated backlog can be unrepresentative. Comparisons should present distributions and uncertainty intervals of each criterion across documents and runs, rather than only a best example. Privacy and confidentiality constraints may prohibit uploading sensitive organizational SRS content to external systems; in these settings, local models or redacted inputs require separate performance reporting.

6.6 Worked example: an illustrative university inventory requirement

The following is a synthetic worked example for explaining the coding logic; the wording is not quoted from the PURE university inventory SRS, and the example is not an experimental test or case-study validation. Hypothetical source: R-INV-01 "An authorized inventory officer shall record an asset transfer between custodians"; R-INV-02 "Every transfer shall retain the old custodian, new custodian, transfer date and approving officer in an audit log."

Table 4. Fictional worked example from SRS obligations to stories, acceptance criteria and source links.

Stage	Illustrative content / adjudication
Source obligations	O1: authorized officer can record transfer (R-INV-01). O2: old/new custodian, date and approver are logged (R-INV-02).
Candidate generated story	US-01: As an inventory officer, I want to record an asset transfer between custodians so that the asset register remains up to date.
Acceptance criterion A1	Given an authorized inventory officer, when an asset is transferred, then the new custodian is saved. (Supports O1.)
Acceptance criterion A2	When the transfer is confirmed, then previous custodian, new custodian, date and approving officer are retained. (Supports O2.)
Traceability map	US-01 / A1 → R-INV-01. A2 → R-INV-02. Reviewers inspect the exact span, not the ID alone.
Unsupported addition flagged	"Automatically email both custodians" is NOT stated in either hypothetical source; mark as proposed/unsupported, not as an approved acceptance criterion.
Illustrative result	2/2 example obligations have a justified mapping; this fraction is a pedagogical calculation, not a measured research result.

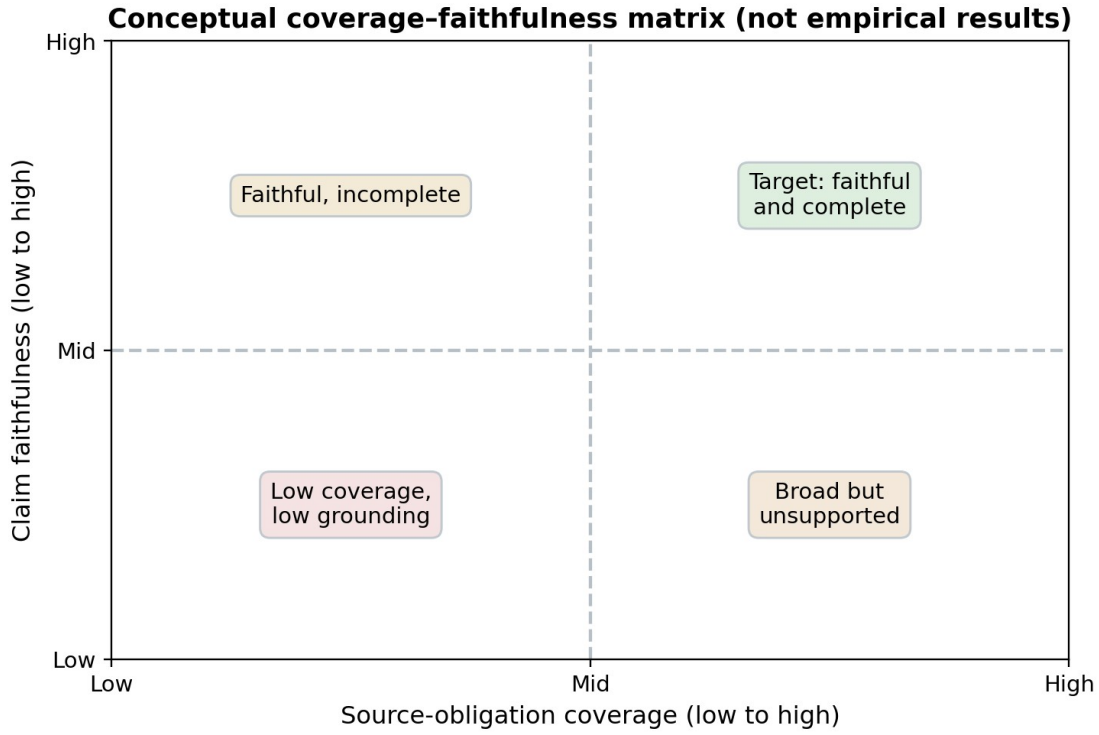


Figure 4. Theoretical coverage–faithfulness trade-off matrix; no empirical cases are plotted.

Table 6. Operational measures proposed for an SRS-to-story benchmark.

Construct	Unit	Proposed operational measure	Common invalid substitute

Source coverage	Atomic SRS obligation	Supported obligations represented / applicable obligations	Story count or topical similarity
Claim faithfulness	Atomic story claim	Supported, contradicted, not stated, indeterminate; source span	LLM self-declared confidence
Traceability	Source–story link	Expert-verified precision/recall/F1; missing links	Presence of an ID string
Story quality	User story	QUS/INVEST rubric, blinded expert rating	Correct "As a…" syntax alone
Testability	Acceptance criterion	Observable conditions, boundaries, constraint evidence	Generated test count
Stability	Model-prompt-document run	Variance and confidence interval across runs	Single best output
Human effort	Analyst correction	Minutes; accepted changes; disagreement frequency	Model response latency alone

7. Research Gaps and Testable Hypotheses

7.1 Joint optimization of coverage and faithfulness

A defensible open research problem is not whether LLMs can produce stories or score their style. It is whether a reproducible generation workflow can jointly increase valid coverage of original requirements and reduce unsupported additions. H1: under a fixed document, model and token budget, prompting with mandatory source-span evidence will lower the proportion of unsupported claims relative to a direct-generation baseline. H2: a decomposition-first pipeline will increase correctly covered atomic obligations, but may increase story count or editorial cost. Both hypotheses require direct expert adjudication of claims and requirements.

7.2 Effect of prompt structure without confounding

Many prompt studies compare techniques while changing additional contextual information, model settings or output constraints. Controlled SRS experiments should isolate the effect of prompt structure. A minimal factorial study could use the same SRS input, model family, token budget and reviewer protocol under: P0 direct prompt; P1 constrained structured output; P2 atomic requirement decomposition; and P3 source-span justification with a verification pass. Models should be repeated across multiple independent SRS documents and runs; "best prompt" should be reported per metric, not assumed universal.

7.2.1 Experimental specification for H1 and H2 (future research only)

The H1/H2 claims are hypotheses for a future experiment, not findings of this review. A minimum follow-up design will use at least three independent SRS documents from distinct applications (a university inventory document can be one pilot case), a preregistered obligation annotation guide, and the same model/version/decoding budget across four prompting conditions (P0–P3). For each document × prompt, use at least five independent stochastic generations and log seeds/settings when available. The primary

endpoints will be expert-adjudicated claim unsupported-addition rate (H1) and obligation coverage proportion (H2); secondary endpoints will include trace-link F1, story count, human correction time and reviewer agreement. Compare within-document paired results, report uncertainty (e.g., bootstrap confidence intervals across documents), and treat document as a clustering unit; any Wilcoxon/paired test would be exploratory with only three documents. Do not infer causality from cross-paper comparisons; a confirmatory hypothesis test requires a prespecified sample-size/power analysis and more documents than this minimum pilot.

Table 7. Suggested experimental conditions for direct SRS-to-story evaluation.

Condition	Control/prompt change	Expected benefit to test	Potential cost or failure
P0: Direct	One-step baseline; story format only	Low latency and minimal setup	Omissions and unsupported features
P1: Structured	Schema, role list, constraints, explicit no-invention instruction	Cleaner outputs and stable extraction	Rigid templates may miss nuance
P2: Decompose	Atomic source obligations before story generation	Better obligation coverage	Errors in atomization propagate
P3: Cite & verify	Require source-span links and claim-level verification	Fewer unsupported claims; auditable links	Higher token use and correction effort

7.3 Human–LLM evaluator calibration

LLM-as-judge scoring can be convenient, but automatic ratings may be sensitive to the model, prompt and reference answer. Existing research already studies human–model agreement in story evaluation [9,15]. A stronger SRS-grounded study should calibrate model judgments separately for style, source support, link correctness and coverage. It should report sample sizes, blinded disagreements, reviewer training, adjudication and confidence intervals. The objective is not to replace expert judgment, but to define what can be screened automatically without concealing disagreements.

7.4 Generalization across documents, languages and institutions

A single university inventory SRS could support a detailed pilot but not a multi-case generalization claim. Replication should use at least three genuinely independent SRSs spanning domains, lengths, documentation quality and role structures. Public corpora such as PURE [21] can be used where license and versioning permit. The independently collected Dalpiaz user-story corpus can inform style and task distribution, but is not a ground-truth paired output for PURE documents. For cross-language systems, translation quality and role interpretation require dedicated assessment rather than reusing English-only quality rules without validation.

7.5 Reproducibility, data governance and release artifacts

A credible study should publish (where permitted) document IDs, extraction rules, source-atom annotations, story prompts, generation settings, run logs, versioned outputs, evaluated claim–source links and the reviewer adjudication record. Commercial SRS data may need secure, limited-access replication packages. Privacy, licensing and institutional approval should be discussed explicitly. Reproducibility includes negative and unsuccessful generations, not only accepted stories.

8. Discussion and Implications

8.1 What practitioners can conclude

LLMs can assist with a first backlog draft, story rewriting and preparation of acceptance criteria, especially when the input document is long and stakeholder review time is limited. Yet the evidence does not justify autonomous replacement of requirements approval. A safe workflow treats every generated story as a candidate artifact that requires review against its originating obligation and relevant non-functional constraints. An explicit "unsupported or proposed" label is more useful than silently incorporating plausible extensions as approved requirements.

8.2 What researchers should stop conflating

Three distinctions deserve particular attention. First, generation quality is not evaluator agreement: a model may provide fluent text that both humans and LLM judges like while losing an original constraint. Second, coverage is not correctness: a higher recall of candidate requirements can coincide with more unsupported additions, as indicated by the differing recall and F1 outcomes in missing-requirement studies [14]. Third, local grounding is not global document completeness: an atomic proposition can be supported by its cited passage even when a separate source obligation has been omitted [13]. Treating all three as "accuracy" destroys interpretability.

8.3 Implications for AI-assisted Agile practice

Teams should maintain version-controlled source IDs, a stable list of authorized roles and a clear process for approving features that the model infers rather than reads from the SRS. The model can be instructed to propose questions when a business rule is ambiguous, but proposed answers must not become accepted stories without stakeholder approval. A traceability dashboard that flags unlinked source requirements and unsupported output claims can support focused human review. For high-risk domains, non-functional constraints and audit trails deserve explicit checkpoints.

8.4 Contribution of the evidence synthesis

The scholarly contribution of this review is to **organize existing evidence around the exact transformation unit and the evaluation construct**, and to propose a falsifiable, operational protocol for source-grounded SRS-to-story work. It is not a new trained model, a benchmark performance claim or a pooled meta-analysis. The framework will acquire empirical status only after independent annotations, controlled multi-case comparisons and results are collected and analyzed.

9. Interpretation and Threats to Validity

The study corpus spans diverse source artifacts, target outputs, domains, prompt structures, and evaluation measures. This heterogeneity supports a structured comparison of evidence constructs rather than pooled effect estimation. Findings from simulated interviews, low-code development projects, issue titles, goal models, or story refinement must not be transferred directly to full-document SRS transformation without matching source and output definitions. Evaluations based on small, homogeneous samples or contextual practitioner feedback should be interpreted in relation to their original study settings.

Measurements of prose fluency, user-story quality, and evaluator agreement do not alone establish fidelity to the originating requirements. Stronger inferences require traceable source spans, explicit

annotation units, and independent adjudication of omissions and unsupported additions. Accordingly, this review distinguishes demonstrations of feasibility from evidence of end-to-end obligation preservation, and treats secondary literature as conceptual support rather than an empirical replicate.

The proposed evaluation protocol is a methodological synthesis of reported work. Its metrics and hypotheses are intended to support future controlled empirical comparisons; the illustrative example is not presented as an executed experiment. The review preserves original denominators and avoids generalizing task-specific performance beyond the documented source studies.

10. Conclusion

Automated transformation of software requirements specifications into Agile user stories is feasible and practically attractive, but its reliability cannot be inferred from fluent text or positive ratings alone. The available research demonstrates useful document-to-story workflows, criterion-dependent prompting outcomes and promising human–LLM quality-assessment methods. It also shows that out-of-scope generation, missing obligations and imperfect evaluator agreement are real concerns. We therefore propose a source-grounded evaluation protocol that separately measures requirement coverage, claim faithfulness, bidirectional traceability, story quality, acceptance-test readiness, stability and human correction cost. Future studies should benchmark controlled prompts across independent SRS documents, use blinded expert annotation, publish linked provenance, and report both omissions and unsupported additions. These steps can turn plausible generated backlogs into auditable engineering artifacts.

Declarations and Supporting Materials

Authorship: Zaher Fadhil Raham and Sura Salah.

Data and materials availability: A curated companion package contains the 181-record bibliographic candidate index, the documented identification counts, DOI/publisher links, the Scopus abstract-source provenance ledger and reusable coding templates. The working Google Drive master preserves reviewer-specific decisions separately. Raw proprietary database exports, publisher PDFs, licensed abstract texts and unapproved reviewer judgments are excluded from the public package. A citable GitHub/Zenodo release URL and DOI should be inserted only after the repository and archival deposit have been completed.

AI assistance disclosure: AI-supported tools were used for bibliographic organization, candidate prioritization, language editing, and preparation of the working evidence register. Such outputs were kept separate from human reviewer decisions and were not used as independent empirical observations. Scientific interpretations and final editorial responsibility belong to the authors.

Reporting scope: The review provides PRISMA-informed, source-documented identification and a task-aware synthesis of inspectable studies. Imported bibliographic entries, deduplicated candidate records, contextual references and empirically assessed outputs are distinct units and are not equated. The proposed framework and worked example are methodological, not findings from a new experiment.

Appendix A. Bibliographic search archive

The identification archive includes one Scopus export split into files of 50 and 43 entries, 28 IEEE Xplore entries and 86 ScienceDirect citation entries. Cross-source comparison identified 26 duplicate entries,

leaving 181 distinct bibliographic candidates. The public companion package lists title, year, DOI or publisher URL, source group and abstract-provenance status without redistributing third-party abstract text. It also contains a reusable coding form and a machine-checkable identification summary. A suggested repeatable topic query is ("user story" OR "user stories") AND ("large language model" OR "LLM" OR "generative AI") AND ("requirements engineering" OR "software requirements" OR "SRS"); this template is not represented as the exact executed historical query.

Appendix B. Extraction form and reviewer decision rubric

Extraction domain	Required fields
Bibliographic identity	DOI; title; publication/online year; version lineage
Task direction	Source input → generated artifact; classify E1, E2, E3 or E4
Study design	Document count, domain, models, prompt templates, settings, comparator
Evaluation	Units, numerator/denominator, metrics, human raters, agreement
Source-grounding	Requirement IDs, obligation ledger, source spans, unsupported claim count
Risk / quality	Dataset/sample; comparator; reviewer independence; reproducibility
Reviewer decision	Include / Exclude / Unclear; exact exclusion reason; page-level evidence
Adjudication	R1 and R2 judgments independently entered, reconciled and time-stamped

References

[1] G. Lucassen, F. Dalpiaz, J. M. E. M. van der Werf, and S. Brinkkemper, “Improving agile requirements: The Quality User Story framework and tool,” Requirements Engineering, vol. 21, pp. 383–403, 2016. https://doi.org/10.1007/s00766-016-0250-x

[2] T. Sporsem, T. Dingsøyr, and K.-J. Stol, “User stories as boundary objects in agile requirements engineering: A theoretical literature review,” Journal of Systems and Software, vol. 233, art. 112693, 2026. https://doi.org/10.1016/j.jss.2025.112693

[3] H. Cheng et al., “Generative AI for Requirements Engineering: A Systematic Literature Review,” Software: Practice and Experience, vol. 56, no. 2, pp. 141–170, 2026. https://doi.org/10.1002/spe.70029

[4] A. Hemmat et al., “Research directions for using LLM in software requirement engineering: A systematic review,” Frontiers in Computer Science, vol. 7, art. 1519437, 2025. https://doi.org/10.3389/fcomp.2025.1519437

[5] K. Huang, F. Wang, Y. Huang, and C. Arora, “Prompt Engineering for Requirements Engineering: A Literature Review and Roadmap,” in 2025 IEEE International Requirements Engineering Conference Workshops, 2025. https://doi.org/10.1109/REW66121.2025.00081

[6] E. Wolf, A. Trendowicz, and J. Siebert, “Quality assessment of software requirements using artificial intelligence methods: A systematic literature review,” Information and Software Technology, vol. 191, art. 107979, 2026. https://doi.org/10.1016/j.infsof.2025.107979

[7] J. Mucha, A. Kaufmann, and D. Riehle, “A systematic literature review of pre-requirements specification traceability,” Requirements Engineering, vol. 29, pp. 119–141, 2024. https://doi.org/10.1007/s00766-023-00412-z

[8] T. Rahman, Y. Zhu, L. Maha, C. K. Roy, B. Roy, and K. Schneider, “Take Loads Off Your Developers: Automated User Story Generation Using Large Language Model,” in IEEE International Conference on Software Maintenance and Evolution (ICSME), 2024. https://doi.org/10.1109/ICSME58944.2024.00082

[9] G. Quattrocchi, L. Pasquale, P. Spoletini, and L. Baresi, “Can LLMs Generate User Stories and Assess Their Quality?,” IEEE Transactions on Software Engineering, vol. 52, no. 5, pp. 1773–1790, 2026. https://doi.org/10.1109/TSE.2026.3670612

[10] T. Spijkman, B. Molenkamp, S. Beudeker, S. Overbeek, and F. Dalpiaz, “LLM-Assisted Requirements Engineering in Agile MDD: Industry Insights and Validation,” in IEEE International Requirements Engineering Conference (RE), pp. 366–377, 2025. https://doi.org/10.1109/RE63999.2025.00043

[11] G. Paiva, E. D. Canedo, and G. P. Rocha Filho, “From issue titles to requirements: An empirical study of large language models and prompt engineering strategies,” Requirements Engineering, vol. 31, art. 7, 2026. https://doi.org/10.1007/s00766-026-00462-z

[12] Y. Li, J. W. Keung, Z. Yang, X. Ma, J. Zhang, and S. Liu, “SimAC: Simulating agile collaboration to generate acceptance criteria in user story elaboration,” Automated Software Engineering, vol. 31, art. 55, 2024. https://doi.org/10.1007/s10515-024-00448-7

[13] R. S. Ayon, R. Fahmi, S. Biswas, and S. Ahmed, “ReqGenX: An Empirical Study of Atomic Decomposition, Artifact Regeneration, and Reconstruction for Legacy SRS Documents,” in 20th International Symposium on Empirical Software Engineering and Measurement, LIPIcs, vol. 394, art. 35, 2026. https://doi.org/10.4230/LIPIcs.ESEM.2026.35

[14] V. Kabadi, A. Wuntakal, N. Gutha, L. Bellarykar, X. B. D. Le, P. Thongtanunam, and C. Treude, “Can LLMs Identify Missing Elements in Requirements Elicitation? An Empirical Evaluation Using Production-Derived User Stories in an Industry Setting,” in 20th International Symposium on Empirical Software Engineering and Measurement, LIPIcs, vol. 394, art. 82, 2026. https://doi.org/10.4230/LIPIcs.ESEM.2026.82

[15] E. Hernández-Agüero, C. Quesada-López, and J. P. Chaves-Sánchez, “Assessing and refining user stories using large language models: A replication study in a real-world organizational software-development setting,” Science of Computer Programming, art. 103575, online 2026. https://doi.org/10.1016/j.scico.2026.103575

[16] N. Alturayeif, I. Ahmad, and J. Hassine, “TraceLLM: Leveraging large language models with prompt engineering for enhanced requirements traceability,” Requirements Engineering, vol. 31, art. 6, 2026. https://doi.org/10.1007/s00766-026-00460-1

[17] T. Zhu, L. C. Cordeiro, and Y. Sun, “REQINONE: A Large Language Model-Based Agent for Software Requirements Specification Generation,” in IEEE International Requirements Engineering Conference (RE), 2025. https://doi.org/10.1109/RE63999.2025.00054

[18] L. Pasquale, A. Ragone, E. Piemontese, and A. A. Darban, “Exploring the Use of LLMs for Requirements Specification in an IT Consulting Company,” in IEEE International Requirements Engineering Conference (RE), 2025. https://doi.org/10.1109/RE63999.2025.00045

[19] A. C. Tricco et al., “PRISMA Extension for Scoping Reviews (PRISMA-ScR): Checklist and Explanation,” Annals of Internal Medicine, vol. 169, pp. 467–473, 2018. https://doi.org/10.7326/M18-0850

[20] M. L. Rethlefsen et al., “PRISMA-S: An extension to the PRISMA statement for reporting literature searches in systematic reviews,” Systematic Reviews, vol. 10, art. 39, 2021. https://doi.org/10.1186/s13643-020-01542-z

[21] A. Ferrari, G. O. Spagnolo, and S. Gnesi, “PURE: A Dataset of Public Requirements Documents,” in IEEE International Requirements Engineering Conference (RE), pp. 502–505, 2017. https://doi.org/10.1109/RE.2017.29

[22] N. D. Bandi and K. A. Demir, “Requirements Engineering with Generative AI: A Systematic Literature Review of Using AI Agents for Requirements Engineering Automation,” Automation, vol. 7, art. 169, 2026. https://doi.org/10.3390/automation7050169

[23] M. Ataei et al., “Elicitron: A large language model agent-based simulation framework for design requirements elicitation,” Journal of Computing and Information Science in Engineering, 2025. https://doi.org/10.1115/1.4067388

[24] L. T. Maha, “Large Language Models for Agile Software Engineering: From Textual Artifacts to Design Models,” Master’s thesis, University of Saskatchewan, 2026. https://harvest.usask.ca/items/d0652cc8-5f8d-45a4-997a-edacc504c7c1

[25] M. A. Zadenoori, J. Dąbrowski, W. Alhoshan, L. Zhao, and A. Ferrari, “Large Language Models (LLMs) for Requirements Engineering (RE): A Systematic Literature Review,” arXiv:2509.11446, 2025 (preprint). https://arxiv.org/abs/2509.11446

[26] M. J. Page et al., “The PRISMA 2020 statement: An updated guideline for reporting systematic reviews,” BMJ, vol. 372, n71, 2021. https://doi.org/10.1136/bmj.n71